

SASBO: Sparse Attack via Stochastic Binary Optimization

Yihan Meng, Weitao Li, and Lin Shang(✉)

State Key Laboratory for Novel Software Technology, Department of Computer Science and Technology, Nanjing University, Nanjing, China
{mengyihan,liweitao}@smail.nju.edu.cn, shanglin@nju.edu.cn

Abstract. Deep Neural Networks have shown vulnerability to sparse adversarial attack, which involves perturbing only a limited number of pixels. Identifying the coordinates requiring perturbation in sparse attacks poses a significant computational challenge. Existing solutions predominantly rely on heuristic methods or relax the ℓ_0-norm to the ℓ_1-norm. In this paper, we present an efficient algorithm for conducting sparse attacks. Our algorithm factorizes the perturbation at each pixel to the product of the perturbation coordinates and the perturbation magnitudes and then optimizes them alternately. We reformulate the ℓ_0-norm as a stochastic binary optimization problem, assuming that each pixel's perturbation status is associated with a stochastic binary variable. This stochastic binary variable follows a Bernoulli distribution, with a parameter value that ranges from 0 to 1, signifying the probability of pixel disturbance. To tackle this stochastic binary optimization challenge, we employ an unbiased gradient estimator known as Augment-Reinforce-Merge (ARM). Once the perturbed coordinates are determined, we optimize the perturbation magnitudes with gradient descent. Furthermore, we incorporate a binary search algorithm to eliminate redundant pixels to enhance sparsity. Comprehensive experiments demonstrate the superiority of our proposed method over several state-of-the-art sparse attack methods.

Keywords: Sparse Attack · Stochastic Binary Optimization · Augment-Reinforce-Merge · Sparsity Enhancement

1 Introduction

Deep Neural Networks (DNNs) have made remarkable strides in various computer vision tasks, such as image classification [10,24], facial expression recognition [20], and image retrieval [9]. Despite their notable achievements, DNNs have been found to exhibit vulnerability to adversarial examples [2,8,19]. Adversarial examples are meticulously crafted images that contain subtle, malicious alterations, aiming to deceive the targeted model while remaining visually indistinguishable from the original, clean images. The rapid development of adversarial attack has given rise to increased concerns about the security of DNNs, especially those deployed in real-world applications [12]. Consequently, research

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2024
D.-N. Yang et al. (Eds.): PAKDD 2024, LNAI 14645, pp. 117–129, 2024.
https://doi.org/10.1007/978-981-97-2242-6_10

on adversarial attack holds significant importance, given that these adversarial examples are invaluable instruments for uncovering the vulnerabilities of DNNs, illuminating their inner mechanisms, and establishing a foundation for enhancing robustness.

In the realm of adversarial perturbations, the predominant consideration revolves around the constraint of ℓ_p-norm distance, which typically falls into two distinct categories: dense attacks and sparse attacks. Dense attacks adhere to the constraints of either the ℓ_2 or ℓ_∞-norm [8,19]. In contrast, sparse attacks are governed by the ℓ_0-norm [14,15] constraint.

Sparse attacks pose a greater challenge in comparison to dense attacks. Such a circumstance arises from the fact that generating adversarial perturbations under the ℓ_0-norm constraint is an NP-hard problem. To address this problem, a multitude of methods have been proposed, broadly falling into two categories. The first category is to predetermine the perturbed pixels artificially or to find the pixels to be perturbed by some heuristic search methods. Nonetheless, there is no assurance that these heuristic methods can pinpoint the optimal perturbation coordinates, as heuristic search strategies are prone to get trapped in local minima and may inadvertently introduce redundant perturbations. The second category involves the relaxation or transformation of ℓ_0-norm constraint, followed by subsequent problem optimization. However, these methods have the high computational complexity and the complex algorithm implementation.

In this paper, we propose a novel sparse adversarial attack approach **SASBO** (**S**parse **A**ttack via **S**tochastic **B**inary **O**ptimization). Initially, we factorize each perturbation into the product of the perturbation magnitudes and the perturbation coordinates. Subsequently, we alternately optimize the coordinates and the magnitudes in each iteration. Regarding the perturbation coordinates, their values dictate whether the corresponding pixel can be perturbed, where a value of 1 indicates permission and 0 denotes exclusion. Because of the binary nature of the coordinates, we model it as a stochastic binary optimization problem, assuming adherence to a Bernoulli distribution. The parameters's values represent the probability of the gate being activated. Consequently, optimizing the coordinates will be formulated as a stochastic binary optimization problem, which can be tackled by the unbiased gradient estimator Augment-Reinforce-Merge (**ARM**) [21]. Especially, ARM is noteworthy for its impartiality, low variance, and computational efficiency, which enables our method to optimize the perturbation coordinates with precision and efficiency. Regarding perturbation magnitudes, we utilize the gradient descent method for optimization. Furthermore, we convert the perturbation coordinates from continuous values to discrete values by using a dynamic threshold strategy. To enhance sparsity, we employ a binary search algorithm to determine the minimum number of pixels required to generate each adversarial example. Finally, we conduct extensive experiments on two benchmark datasets, CIFAR-10 [11] and ImageNet [5]. These experiments empirically validate the effectiveness and performance of our proposed method.

In summary, our work makes three primary contributions:

- We introduce an innovative sparse attack method, regarding it as a stochastic binary optimization problem and optimizing it with ARM.

- We utilize a binary search algorithm to enhance the sparsity of generated adversarial examples by eliminating redundant pixels.
- We demonstrate our method's outstanding performance in untargeted and targeted attacks through comprehensive experiments.

2 Related Work

The generation of sparse adversarial examples for image classification tasks has been the subject of extensive research in recent years. Existing adversarial attack methods can be primarily categorized into two distinct groups: heuristic strategies and optimized strategies.

Heuristic Strategies. One Pixel Attack [18] leverages the Differential Evolution algorithm to seek an extremely sparse configuration, where only one pixel is perturbed to deceive the target model. JSMA [16] and its extensions select influential pixels based on a saliency map, while CornerSearch [3] employs a heuristic approach that involves traversing all pixels and selecting a subset for perturbation. Additionally, GreedyFool [6] iteratively identifies the most suitable pixels for modification based on gradient information and subsequently applies a greedy strategy to drop as many less critical pixels as possible. However, it is essential to note that these heuristic-based methods share common challenges. Due to the nature of the heuristic search, they are susceptible to local optima, potentially resulting in unnecessary and redundant perturbations.

Optimized Strategies. SparseFool [15] converts the ℓ_0 problem into an ℓ_1 problem. PGD_0 [3] proposes to project the adversarial noise generated by PGD [14] to the ℓ_0-ball to achieve the ℓ_0-version PGD. An ADMM-based method proposed in [22] aims to optimize the sparse attack problem by decoupling the ℓ_0-norm and the adversarial loss. ℓ_1-APGD [4] converts the ℓ_0-norm constrain into the intersection of the ℓ_1-ball and $[0,1]^d$ box constrain. Subsequently, it derives the appropriate steepest descent step for the intersection, which motivates an adaptive sparsity of the chosen descent direction. SAPF [7] reformulates the sparse adversarial attack problem as a mixed integer programming problem, to jointly optimize the binary selection factors and continuous perturbation magnitudes of all pixels, with a cardinality constraint that explicitly controls the degree of sparsity. Homotopy-Attack [23] addresses the sparse attack problem by proposing a homotopy algorithm. In each iteration, Homotopy-Attack optimizes the ℓ_0-regularized adversarial loss by leveraging the nonmonotone Accelerated Proximal Gradient Method for nonconvex programming. Additionally, it incorporates an optional post-attack step designed to escape undesirable local minima.

3 Methods

3.1 Problem Definition

We denote the source image as x and its ground-truth label as y. An input x will be classified as $\arg\max f_c(x) = y$, where f is the target model and $f_c(x)$ is the

output logit value for class c. An adversarial example $x^{adv} = x + \boldsymbol{\delta}$ is generated by adding perturbations to the source image.

The untargeted attack aims to ensure $\arg\max f_c(x) \neq y$. Furthermore, targeted attack aims to guarantee $\arg\max f_c(x) = y_{adv}$, where y_{adv} is a designated adversarial label. Moreover, the adversarial perturbation $\boldsymbol{\delta}$ should be small enough to guarantee the visual similarity between the adversarial sample and the source image. In this paper, we employ ℓ_0-norm, which means that adversarial perturbations can only be added to a few pixels, known as *sparse attack*. Thus, the goal of the adversarial attack can be formulated as:

$$\min_{\delta} \|\boldsymbol{\delta}\|_0 + \lambda \mathcal{L}\left(f(x+\boldsymbol{\delta}), y_t\right), \quad \text{s.t.} \quad x + \boldsymbol{\delta} \in [0,1], \tag{1}$$

where $\mathcal{L}$ is the loss function, such as the widely used C&W loss [2], and λ serves as a weighting factor for the regularization.

However, the sparse attack is an NP-hard problem, which can not be solved analytically within polynomial time, primarily due to the non-differentiability of ℓ_0-norm. Previous approaches have employed stochastic search strategies or heuristic methods to determine the perturbed coordinates. In contrast, we propose to transform the discrete optimization problem of perturbed coordinates into a continuous optimization problem, and then alternately optimize the perturbation coordinates and perturbation magnitudes.

3.2 Sparse Adversarial Attack via Stochastic Binary Optimization

We first factorize the perturbation $\boldsymbol{\delta}$ into the following two parts:

$$\boldsymbol{\delta} = \mathbf{M} \odot \mathbf{E}. \tag{2}$$

$\mathbf{M} \in \{0,1\}^N$ denotes the vector of perturbed coordinates, which should be a sparse matrix. $\mathbf{E} \in \mathbb{R}^N$ denotes the vector of perturbation magnitudes. $\odot$ denotes element-wise product. Since $\mathbf{E}$ is continuous, while $\mathbf{M}$ is discrete, it is unfeasible to jointly optimize them using any existing continuous solvers, such as stochastic gradient descent.

Optimize Perturbed Coordinates M with ARM. Following the previous factorization, and given $\mathbf{E}$, we can reformulate the goal of the sparse adversarial attack concerning $\mathbf{M}$ as follows:

$$\min_{M} \|\mathbf{M} \odot \mathbf{E}\|_0 + \lambda \mathcal{L}\left(f(x + \mathbf{M} \odot \mathbf{E}), y_t\right), \quad \text{s.t.} \quad \mathbf{M} \in \{0,1\}^N, \tag{3}$$

Note that the Eq. (3) remains computationally intractable since $m_j \in \mathbf{M}$ is a discrete variable. Therefore, we must consider further transformations of the variable $\mathbf{M}$. These transformations should enable us to efficiently optimize Eq. (3) in a continuous manner while maintaining the integrity of the ℓ_0-norm constraints.

The value of $m_j \in \mathbf{M}$ is a discrete value 0 or 1, corresponding to a binary variable that denotes whether the pixel can be perturbed. The ℓ_0-norm represents the number of variables with perturbed permission. To relax this discrete constraint, we assume m_j adheres to Bernoulli distribution with parameter $\pi_j \in [0,1]$, such as $m_j \sim \text{Bern}\left(m_j; \pi_j\right)$. After this transformation, the value of m_j is continuous,

and the ℓ_0-norm corresponds to the variable π_j, which determines the probability that gate m_j is equal to one. In addition, according to stochastic variational optimization [1], given any function $\mathcal{F}(\boldsymbol{z})$ and any distribution $q(\boldsymbol{z})$, the following inequality holds

$$\min_{\boldsymbol{z}} \mathcal{F}(\boldsymbol{z}) \leq \mathbb{E}_{\boldsymbol{z}\sim q(\boldsymbol{z})}[\mathcal{F}(\boldsymbol{z})], \tag{4}$$

which means the minimum value of a function is upper bounded by the expectation of the function. With Eq. (4) and substituting $\mathbf{M} \sim \text{Bern}(\mathbf{M}; \pi)$ into (3), problem (3) can be rewritten as:

$$\min_{\boldsymbol{\pi}} \mathcal{R}(\boldsymbol{\pi}) = \mathbb{E}_{M\sim \text{Ber}(M;\boldsymbol{\pi})}\left[\lambda \mathcal{L}\left(f(x+\mathbf{M}\odot\mathbf{E}), y_t\right)\right] + \sum_{j=1}^{N} e_j \cdot \pi_j \tag{5}$$

The second term represents sparse constraint after relaxation, where π_j is the perturbed probability of pixel at m_j, and e_j is the perturbation size of the pixel at m_j. The problem is an expectation minimization problem characterized by introducing parameters π. Note that the second term is differentiable and related to the new parameters. However, the first term remains problematic since the expectation over numerous binary random variables M is intractable and does not allow for efficient gradient-based optimization.

In order to minimize the new problem, we can use the Augment-Reinforce-Merge (ARM) [21] gradient estimator, which is proposed for the optimization of binary latent variables problem. Denote $\sigma(\phi) = 1/(1 + exp(-\phi))$ as a sigmoid function, which shares the same value range as π. $\mathbf{1}_{[\cdot]}$ is an indicator function that equals to one if the argument is true and zero otherwise

Theorem 1 *(ARM). For a vector of V binary random variables* $\boldsymbol{z} = (z_1, \ldots, z_V)^T$*, the gradient of*

$$\mathcal{E}(\boldsymbol{\phi}) = \mathbb{E}_{\boldsymbol{z}\sim\prod_{v=1}^{V}\text{Bernoulli}(z_v;\sigma(\phi_v))}[f(\boldsymbol{z})] \tag{6}$$

with respect to $\phi = (\phi_1, \ldots, \phi_V)^T$*, the logits of the Bernoulli probability parameters can be expressed as*

$$\nabla_{\boldsymbol{\phi}}\mathcal{E}(\boldsymbol{\phi}) = \mathbb{E}_{\boldsymbol{u}\sim\prod_{v=1}^{V}\text{Uni}(u_v;0,1)}\left[\left(f\left(\mathbf{1}_{[\boldsymbol{u}>\sigma(-\boldsymbol{\phi})]}\right) - f\left(\mathbf{1}_{[\boldsymbol{u}<\sigma(\boldsymbol{\phi})]}\right)\right)\left(u - \frac{1}{2}\right)\right], \tag{7}$$

where $\mathbf{1}_{[\boldsymbol{u}>\sigma(-\boldsymbol{\phi})]} := \left(\mathbf{1}_{[u_1>\sigma(-\phi_1)]}, \ldots, \mathbf{1}_{[u_V>\sigma(-\phi_V)]}\right)^T$.

With parameter $\pi_j \in [0, 1]$ as $\sigma(\phi_j)$, problem (5) can be reformulated as

$$\min_{\boldsymbol{\phi}} \mathcal{R}(\boldsymbol{\phi}) = \mathbb{E}_{M\sim\text{Ber}(M;\sigma(\boldsymbol{\phi}))}\left[\lambda\mathcal{L}\left(f(x+\mathbf{M}\odot\mathbf{E}), y_t\right)\right] + \sum_{j=1}^{N} e_j \cdot \sigma(\phi_j) \tag{8}$$

According to Theorem 1, we can compute the gradient of Eq. (8) with respect to ϕ by

$$\begin{aligned}\nabla_{\boldsymbol{\phi}}^{ARM}\mathcal{R}(\boldsymbol{\phi}) = &\mathbb{E}_{\boldsymbol{u}\sim\prod_{v=1}^{V}\text{Uni}(\boldsymbol{u};0,1)}\left[\left(f\left(\mathbf{1}_{[\boldsymbol{u}>\sigma(-\boldsymbol{\phi})]}\right) - f\left(\mathbf{1}_{[\boldsymbol{u}<\sigma(\boldsymbol{\phi})]}\right)\right)\left(\boldsymbol{u} - \frac{1}{2}\right)\right] \\ &+ \lambda\sum_{j=1}^{N} e_j \cdot \nabla_{\phi_j}\sigma\left(\phi_j\right)\end{aligned} \tag{9}$$

which is an unbiased and low variance estimator as demonstrated in [21].

After completing the optimization steps mentioned above, we can obtain the parameter ϕ. Each element of ϕ represents the perturbation probability of the corresponding pixel. As a result, $\mathbf{M}$ can be obtained by taking the expectation of $m_j \sim \text{Bern}\left(m_j; \sigma\left(\phi_j\right)\right)$

$$m_j = \mathbb{E}[m_j] = \pi_j = \sigma\left(\phi_j\right) \qquad j = 1, 2, \cdots, N \tag{10}$$

A conventional approach to convert continuous values to discrete values approximately is specifying a threshold τ for partition. It is worth noting that when utilizing a large threshold, the resulting perturbation coordinates matrix $\mathbf{M}$ exhibits excessive sparsity, rendering it ineffective in generating successful adversarial examples for certain images. Conversely, a smaller threshold leads to perturbations that are overly redundant. Sensitivity to the threshold parameter highlights the necessity for a more nuanced and adaptive discretization strategy within our algorithmic framework.

Hence, we opt for a dynamic thresholding approach by determining the threshold τ based on the k-th largest element in the output vector $\sigma\left(\phi\right)$. The parameter k is introduced as a hyperparameter, serving the purpose of regulating sparsity. This strategic selection of the threshold contributes to a nuanced control over the extent of perturbation in the context of adversarial attacks.

$$\tau = \text{argmax}_k(\sigma\left(\phi\right)) \tag{11}$$

Although the upper bound is sufficient for successful sparse attacks, it may not be the minimum number of required pixels for the adversary. For sparser perturbations, it is crucial to minimize the number of perturbed pixels as small as possible. Therefore, we adopt a binary search strategy to achieve better sparsity, and the details are introduced in Algorithm 1.

Optimize Perturbation Magnitudes E by Gradient Descent. Given the perturbation coordinates $\mathbf{M}$, the Eq. (3) associated with $\mathbf{E}$ can be reformulated as follows:

$$\min_E \|\mathbf{M} \odot \mathbf{E}\|_2 + \lambda \mathcal{L}\left(f(x + \mathbf{M} \odot \mathbf{E}), y_t\right), \quad -\epsilon \leq \mathbf{E} \leq \epsilon, \tag{12}$$

where ϵ is the maximal allowable perturbation magnitudes, and we use ℓ_2-norm as the metric to quantify the magnitudes of the perturbations.

When the perturbation coordinates have been determined, we need to calculate the perturbation magnitudes of the corresponding coordinates. Since the sparse matrix $\mathbf{M}$ satisfies the requirement of the sparsity constraint, ℓ_0-norm is no longer required as a constraint for adversarial perturbations in Eq. (12). To ensure the invisibility of the perturbation, we opt to employ the ℓ_2-norm to constrain the dissimilarity between the adversarial examples and the original images.

After the above reformulation, Eq. (12) is akin to the dense adversarial attack formulation and is differentiable. Hence, we can optimize it by utilizing the gradient descent algorithm as follows:

$$\nabla_{\mathbf{E}} = 2 \cdot (\mathbf{M} \odot \mathbf{M} \odot \mathbf{E}) + \lambda \frac{\partial \mathcal{L}\left(f(x + \mathbf{M} \odot \mathbf{E}), y_t\right)}{\partial \mathbf{E}} \tag{13}$$

Algorithm 1. SASBO

Input: source image $\mathbf{x}$, targeted model f.
Parameter: iterative steps N_1, N_2, Bernoulli probability parameter $\sigma(\phi)$, upper bound of pixels k, learning rate η_1, η_2, perturbation threshold ϵ.
Output: adversarial example $\mathbf{x}^{adv}$.

Stage 1: Optimization
$\phi = \mathcal{N}(0, 0.01)$
$\mathbf{E} = 0.01$
while not converged **do**
  Update $\mathbf{M}$ with ARM.
  for $0...N_1$ **do**
    $\boldsymbol{\phi} = \boldsymbol{\phi} - \boldsymbol{\eta_1} \cdot \nabla_{\phi}^{ARM} \mathcal{R}(\boldsymbol{\phi})$
  end for
  $\boldsymbol{\tau} = \text{argmax}_k(\boldsymbol{\sigma}(\boldsymbol{\phi}))$
  $\mathbf{M} = \mathbf{1}_{[\mathbf{m}_j > \tau]}$
  Update $\mathbf{E}$ with gradient descent.
  for $0...N_2$ **do**
    $\mathbf{E} = \mathbf{E} - \boldsymbol{\eta_2} \cdot (2 \cdot (\mathbf{M} \odot \mathbf{E} \odot \mathbf{E}) + \lambda \frac{\partial \mathcal{L}(f(\mathbf{x}+\mathbf{M}\odot\mathbf{E}), y_t)}{\partial \mathbf{E}})$
    $\mathbf{E} = clip(\mathbf{E}, -\boldsymbol{\epsilon}, \boldsymbol{\epsilon})$
  end for
end while

Stage 2: Binary Search
$LB = 1$
$UB = k$
while $UB - LB > 1$ **do**
  $MI = (LB + UB)/2$
  $\tau = \text{argmax}_{MI}(\sigma(\phi))$
  $\mathbf{M} = \mathbf{1}_{[m_j > \tau]}$
  $\mathbf{x}^{adv} = \mathbf{x} + \mathbf{M} \odot \mathbf{E}$
  if $\mathbf{x}^{adv}$ is not adversarial **then**
    $LB = MI + 1$
  else
    $UB = MI$
  end if
end while
return $\mathbf{x}^{adv}$

Finally, we clip the perturbation magnitudes $\mathbf{E}$ to the range $[-\epsilon, \epsilon]$ after each gradient step in order to satisfy perturbation threshold $\boldsymbol{\epsilon}$ constraints.

4 Experiments and Results

In this section, we conduct experiments on CIFAR-10 [11] and ImageNet [5] datasets. We compare our method with several state-of-the-art sparse adversarial attack algorithms, including PGD-$\ell_0 + \ell_\infty$ [3], SparseFool [15], and GreedyFool [6].

Dataset and Classification Model Setting. For CIFAR-10 dataset, following [6] and [3], we employ a pre-trained network in network [13] model with input size of $32 \times 32 \times 3$. For the classification model on the ImageNet dataset, we opt to utilize the pre-trained VGG19 [17] model. To make an accurate comparison, we generate adversarial examples with 2000 images randomly selected from the ImageNet validation set and 5000 images from the CIFAR-10 test set. For the targeted attack, the targeted label is randomly selected and kept consistent for all methods to make a fair comparison.

Parameter Settings. In SASBO, the trade-off hyperparameter λ is set to 1 to balance the adversarial loss and sparsity. As shown in Algorithm 1, we initialize ϕ by random samples from a normal distribution $\mathcal{N}(0, 0.01)$ and set the initial

value of $\mathbf{E}$ to 0.01. During each iteration, $\mathbf{M}$ and $\mathbf{E}$ are updated $N_1 = 5$ and $N_2 = 10$ steps with learning rate $\eta_1 = 0.01$ and $\eta_2 = 0.1$, respectively, so that when one of $\mathbf{M}$ or $\mathbf{E}$ is specified the other can be optimized to the current optimal solution. Besides, the number of upper bound perturbed coordinates k is a key hyper-parameter for our method. We initially run the baseline GreedyFool with 100% ASR and use the average ℓ_0-norm of GreedyFool as the reference value for k. Then, we tune the value of k to ensure successful attacks on all images.

For both targeted attack and non-targeted attacks, we employ the C&W loss function [2] as the adversarial loss function $\mathcal{L}$. As for compared methods, we employ the official implementation and follow default settings in the experiments.

Evaluation Metrics. We report the average ℓ_p norms of generated perturbations, where $p = 0, 1, 2, \infty$, and the attack success rate (ASR) of the experimented methods with both the targeted and non-targeted settings.

4.1 Non-targeted Attack

We evaluate the sparsity and ASR under three different perturbation magnitude thresholds. On both the CIFAR-10 and ImageNet datasets, we compare our methods with recent works: PGD-$\ell_0 + \ell_\infty$, SparseFool, GreedyFool. The non-targeted attack results on both CIFAR-10 and ImageNet datasets are presented in Table 1 and Table 2.

Table 1. Statistics of ASR and average ℓ_p-norms ($p = 0, 1, 2, \infty$) of non-targeted attack on CIFAR-10 dataset.

Threshold	Method	ASR	ℓ_0	ℓ_1	ℓ_2	ℓ_∞
$\epsilon = 255$	PGD-$\ell_0 + \ell_\infty$	100	48.01	38.59	6.05	1.000
	SparseFool	100	22.03	10.61	2.29	**0.808**
	GreedyFool	100	12.29	8.14	1.48	0.986
	SASBO (Ours)	**100**	**3.95**	**3.82**	**1.24**	0.985
$\epsilon = 100$	PGD-$\ell_0 + \ell_\infty$	100	487.82	171.82	8.18	0.389
	SparseFool	100	30.39	9.81	1.71	0.386
	GreedyFool	100	15.56	5.83	1.21	**0.376**
	SASBO (Ours)	**100**	**7.90**	**3.08**	**0.51**	0.382
$\epsilon = 10$	PGD-$\ell_0 + \ell_\infty$	82.72	941.78	45.72	1.88	0.039
	SparseFool	100	315.13	12.25	0.57	0.039
	GreedyFool	100	178.53	6.56	0.49	**0.038**
	SASBO (Ours)	**100**	**144.21**	**5.61**	**0.43**	0.039

For CIFAR-10, as depicted in Table 1, when the perturbation threshold ϵ is 255, it is evident that all methods can achieve a 100% attack success rate. Furthermore, our algorithm exhibits a significant advantage over the other methods

Table 2. Statistics of ASR and average ℓ_p-norms ($p = 0, 1, 2, \infty$) of non-targeted attack on ImageNet dataset.

Threshold	Method	ASR	ℓ_0	ℓ_1	ℓ_2	ℓ_∞
$\epsilon = 255$	PGD-$\ell_0 + \ell_\infty$	100	1315.73	1315.73	114.68	1.000
	SparseFool	100	102.45	49.33	4.56	0.860
	GreedyFool	100	71.65	30.34	3.06	0.867
	SASBO (Ours)	**100**	**50.87**	**24.52**	**1.65**	**0.860**
$\epsilon = 100$	PGD-$\ell_0 + \ell_\infty$	100	1782.94	686.33	16.22	0.385
	SparseFool	100	172.78	45.02	2.96	0.386
	GreedyFool	100	160.46	36.91	2.3	**0.345**
	SASBO (Ours)	**100**	**145.77**	**38.29**	**1.44**	0.379
$\epsilon = 10$	PGD-$\ell_0 + \ell_\infty$	64.92	14750.05	535.25	4.56	0.039
	SparseFool	100	1257.56	39.10	1.04	0.039
	GreedyFool	100	1104.27	38.95	0.84	**0.038**
	SASBO (Ours)	**100**	**1051.24**	**40.86**	**0.57**	0.039

in terms of sparsity. Specifically, the average perturbation number required by our algorithm is only 3.95 pixels to execute a successful attack, while GreedyFool needs to perturb 12.29 pixels, which is approximately 3.11 times higher than ours.

As the perturbation threshold ϵ decreases, executing the attack successfully becomes increasingly challenging. All the methods require more perturbation pixels to perform a successful attack. Nonetheless, our method consistently outperforms the compared methods. For instance, when the perturbation threshold ϵ is 10, PGD-ℓ_0+ℓ_∞ fail to perform non-targeted attack with 100% attack success rate. In contrast, our algorithm maintains a 100% attack success rate. Besides, our algorithm can still achieve the best sparsity, requiring only 144.21 pixels to be perturbed, which is a notable reduction compared to all other algorithms.

For the ImageNet dataset, it can be observed that our method consistently outperforms other methods in most settings. Under three different perturbation threshold settings, our algorithm can achieve a 100% attack success rate with the lowest average perturbation number. When perturbation threshold ϵ is 255, the average ℓ_0 norm of our algorithm is 50.87 pixels, while the average ℓ_0 norm of GreedyFool is 71.65 pixels, which is 1.41x than us.

As evident from the data presented in Table 1 and Table 2, our algorithm can achieve the lowest ℓ_0, ℓ_1, ℓ_2 and ℓ_∞ norm at the same time under most settings, when compared to the other methods. This indicates that our method can select pixels that contribute more adversarial than other pixels, and all the perturbed pixels have been fully used within the allowable perturbation threshold, resulting in sparser results. Moreover, this also illustrates our method can obtain a better approximate optimal for the adversarial attack problem under ℓ_0-norm.

4.2 Targeted Attack

Since SparseFool can only achieve non-targeted attacks, here we compare targeted attack results with PGD-$\ell_0 + \ell_\infty$, GreedyFool on both CIFAR-10 and ImageNet dataset. Table 3 and Table 4 present the experimental results of the targeted attack on the CIFAR-10 and ImageNet datasets.

Table 3. Statistics of attack success rate (ASR) and average ℓ_p-norms ($p = 0, 1, 2, \infty$) of targeted attack on CIFAR-10 dataset.

Threshold	Method	ASR	ℓ_0	ℓ_1	ℓ_2	ℓ_∞
$\epsilon = 255$	PGD-$\ell_0 + \ell_\infty$	100	54.03	32.59	5.54	1.000
	GreedyFool	100	48.99	16.52	2.61	0.992
	SASBO (Ours)	**100**	**14.88**	**13.95**	**1.87**	**0.981**
$\epsilon = 100$	PGD-$\ell_0 + \ell_\infty$	100	461.91	168.24	8.09	**0.389**
	GreedyFool	100	98.46	15.83	3.41	0.391
	SASBO (Ours)	**100**	**29.94**	**11.34**	**1.55**	0.390
$\epsilon = 10$	PGD-$\ell_0 + \ell_\infty$	54.79	955.35	32.17	1.19	0.039
	GreedyFool	100	412.08	14.41	0.79	0.039
	SASBO (Ours)	**100**	**324.14**	**12.62**	**0.68**	**0.039**

As presented in Table 3, when ϵ is 255, we can observe that all compared methods can achieve a 100% attack success rate. Notably, SASBO demonstrates superior sparsity in comparison to the other methods. In detail, the GreedyFool achieves a sparsity of 1.59% (48.99 pixels) on $32 \times 32 \times 3$ CIFAR-10 images with $\epsilon = 255$, while SASBO can achieve a sparsity of 0.48% (14.88 pixels) on $32 \times 32 \times 3$ CIFAR-10 images, which is 69.6% fewer when compared to GreedyFoll. Although achieving a targeted attack becomes more difficult as the perturbation threshold ϵ decreases to 10, SASBO can still maintain a 100% attack success rate. In contrast, the attack success rate of PGD-$\ell_0 + \ell_\infty$ decreases rapidly to 54.79%. Meanwhile, the ℓ_0-norm of PGD-$\ell_0 + \ell_\infty$ increases rapidly. The average perturbation pixel number of PGD-$\ell_0 + \ell_\infty$ increases from 54.03 to 955.35. However, SASBO requires only 324.14 pixels to achieve the targeted attack successfully.

Regarding the ImageNet dataset, our algorithm still performs best in most settings. This also demonstrates the effectiveness of our algorithm. As shown in Table 4, our algorithm achieves a 100% attack success rate under all three settings. The PGD-$\ell_0 + \ell_\infty$ and GreedyFool algorithms achieve 100% attack success rate under perturbation threshold ϵ is 255, and the average perturbation number is 10015.04 pixels and 503.27 pixels respectively. However, our algorithm can achieve the same 100% attack success rate with the lowest average perturbation number, only 272.95 pixels.

The results presented in Tables 3 and 4 illustrate that SASBO is effective in both targeted and non-targeted attacks that both achieve better sparsity.

Table 4. Statistics of attack success rate (ASR) and average ℓ_p-norms ($p = 0, 1, 2, \infty$) of targeted attack on ImageNet dataset.

Threshold	Method	ASR	ℓ_0	ℓ_1	ℓ_2	ℓ_∞
$\epsilon = 255$	PGD-$\ell_0 + \ell_\infty$	100	10015.04	9873.23	113.13	1.000
	GreedyFool	100	503.27	350.41	27.38	**0.935**
	SASBO (Ours)	**100**	**272.95**	**205.87**	**15.33**	0.995
$\epsilon = 100$	PGD-$\ell_0 + \ell_\infty$	100	32842.92	10867.91	60.23	**0.389**
	GreedyFool	100	1196.42	173.76	10.25	0.391
	SASBO (Ours)	**100**	**600.80**	**142.83**	**6.56**	0.390
$\epsilon = 10$	PGD-$\ell_0 + \ell_\infty$	51.02	49516.20	1826.12	8.72	0.039
	GreedyFool	100	6476.42	225.64	3.12	0.039
	SASBO (Ours)	**100**	**4782.61**	**186.49**	**2.61**	**0.039**



(a) Clean (b) SASBO (c) SASBO (d) GreedyF (e) GreedyF

Fig. 1. Visualization of nontargeted attack under perturbation threshold $\epsilon = 10$. The images in each row, from left to right, are clean images, our adversarial example, our perturbation coordinates, GreedyFool's adversarial example, and GreedyFool's perturbation coordinates. In the figures of perturbation coordinates, black pixels denote no perturbation; white pixels denote perturbing all three RGB channels; pure red, green, and blue pixels represent perturbing a single channel. (Color figure online)

4.3 Visualization

For adversarial examples, invisibility serves as a pivotal metric. Figure 1 presents visualizations illustrating instances of non-targeted attacks executed by our algorithm and GreedyFool. Scrutinizing columns 1–2 of Fig. 1, it is almost impossible to figure out the difference between clean images and our adversarial examples, proving that our algorithm can guarantee the invisibility of adversarial examples. Furthermore, as depicted in columns 3 and 5 of Fig. 1, our method perturbs only a single channel, while GreedyFool perturbs all three RGB channels. Consequently, the alterations in perturbed pixels are slight, highlighting that our algorithm ensures both sparsity and imperceptibility concurrently.

5 Conclusion

In this paper, we propose a novel sparse adversarial attack algorithm. Our approach commences by converting the discrete variable representing the perturbed coordinates into a continuous random variable. This transformation allows us to frame the sparse adversarial attack as a stochastic binary optimization problem, which can be efficiently and precisely solved using the ARM method. Experiments conducted on two benchmark datasets show that our method achieves optimal performance in terms of attack success rate and sparsity, among other metrics, when compared to existing state-of-the-art sparse adversarial attack methods. In addition, the results of the visual presentation of the adversarial samples also show that the adversarial samples generated by our method have good adversarial perturbation invisibility. In the future, we will further consider how to incorporate the proposed ideas into group sparsity to help achieve better invisibility.

References

1. Bird, T., Kunze, J., Barber, D.: Stochastic variational optimization. arXiv preprint arXiv:1809.04855 (2018)
2. Carlini, N., Wagner, D.: Towards evaluating the robustness of neural networks. In: SP, pp. 39–57. IEEE (2017)
3. Croce, F., Hein, M.: Sparse and imperceivable adversarial attacks. In: ICCV, pp. 4723–4731. IEEE (2019)
4. Croce, F., Hein, M.: Mind the box: l_1-APGD for sparse adversarial attacks on image classifiers. In: ICML, pp. 2201–2211. PMLR (2021)
5. Deng, J., Dong, W., Socher, R., Li, L., Li, K., Fei-Fei, L.: ImageNet: a large-scale hierarchical image database. In: CVPR, pp. 248–255. IEEE (2009)
6. Dong, X., et al.: GreedyFool: distortion-aware sparse adversarial attack. In: NeurIPS, pp. 11226–11236 (2020)
7. Fan, Y., et al.: Sparse adversarial attack via perturbation factorization. In: Vedaldi, A., Bischof, H., Brox, T., Frahm, J.-M. (eds.) ECCV 2020. LNCS, vol. 12367, pp. 35–50. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-58542-6_3
8. Goodfellow, I.J., Shlens, J., Szegedy, C.: Explaining and harnessing adversarial examples. In: ICLR (2015)
9. Gordo, A., Almazán, J., Revaud, J., Larlus, D.: Deep image retrieval: learning global representations for image search. In: Leibe, B., Matas, J., Sebe, N., Welling, M. (eds.) ECCV 2016. LNCS, vol. 9910, pp. 241–257. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-46466-4_15
10. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: CVPR, pp. 770–778. IEEE (2016)
11. Krizhevsky, A., Hinton, G., et al.: Learning multiple layers of features from tiny images. In: Handbook of Systemic Autoimmune Diseases (2009)
12. Kurakin, A., Goodfellow, I.J., Bengio, S.: Adversarial machine learning at scale. In: ICLR (2017)
13. Lin, M., Chen, Q., Yan, S.: Network in network. In: ICLR (2014)
14. Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A.: Towards deep learning models resistant to adversarial attacks. In: ICLR (2018)

15. Modas, A., Moosavi-Dezfooli, S.M., Frossard, P.: SparseFool: a few pixels make a big difference. In: CVPR, pp. 9087–9096. IEEE (2019)
16. Papernot, N., McDaniel, P., Jha, S., Fredrikson, M., Celik, Z.B., Swami, A.: The limitations of deep learning in adversarial settings. In: EuroSP, pp. 372–387 (2016)
17. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. In: ICLR (2015)
18. Su, J., Vargas, D.V., Sakurai, K.: One pixel attack for fooling deep neural networks. TEVC (2019)
19. Szegedy, C., et al.: Intriguing properties of neural networks. In: ICLR (2014)
20. Yang, H., Ciftci, U.A., Yin, L.: Facial expression recognition by de-expression residue learning. In: CVPR, pp. 2168–2177. IEEE (2018)
21. Yin, M., Zhou, M.: ARM: augment-reinforce-merge gradient for stochastic binary networks. In: ICLR (2019)
22. Zhao, P., Liu, S., Wang, Y., Lin, X.: An ADMM-based universal framework for adversarial attacks on deep neural networks. In: ACM MM. ACM (2018)
23. Zhu, M., Chen, T., Wang, Z.: Sparse and imperceptible adversarial attack via a homotopy algorithm. In: ICML, pp. 12868–12877. PMLR (2021)
24. Zoph, B., Vasudevan, V., Shlens, J., Le, Q.V.: Learning transferable architectures for scalable image recognition. In: CVPR, pp. 8697–8710. IEEE (2018)